

PROGRAMACIÓN AVANZADA

Sesión 21

Archivos de texto

→ NOTAS DE CLASE

Índice

1	1970: cuando la computadora aprendió a tener cajones	2
2	Memoria vs. archivos: el escritorio y los cajones	2
3	¿Qué es un archivo, exactamente?	3
3.1	Verlo con tus propios ojos	3
3.2	Salto de línea	4
4	La biblioteca <code><fstream></code>	4
5	Leyendo un archivo línea por línea	5
6	Escribiendo en un archivo	6
6.1	¿Por qué llamar a <code>close()</code> ?	6
7	Leyendo valores numéricos	6
8	Actividad: agenda de cumpleaños	7
8.1	Formato del archivo	7
8.2	Parte A: genera tu archivo (10 min)	8
8.3	Parte B: las consultas (40 min)	8
9	Ejercicios	8

1970: cuando la computadora aprendió a tener cajones

En 1970, en los laboratorios Xerox PARC en Palo Alto, un equipo de investigadores liderado por Alan Kay se enfrentaba a un problema que ninguna ecuación podía resolver: las computadoras eran intimidantes. Los usuarios tenían que memorizar comandos crípticos, navegar sistemas de archivos sin ningún contexto visual y entender abstracciones que no tenían nada que ver con su vida cotidiana.

La solución fue radical: hacer que la computadora pareciera un escritorio de oficina. Un escritorio con papeles encima, cajones donde guardar documentos, carpetas para organizar los cajones y una papelerita de reciclaje en la esquina. No fue una decisión de diseño gráfico. Fue una decisión pedagógica deliberada: si la gente ya sabe usar un escritorio, puede aprender a usar una computadora.

El nombre de los conceptos no es traducción: *desktop*, *folder*, *file* y *trash* fueron elegidos precisamente porque ya existían en el mundo físico. Hoy, en esta sesión, vas a entender por qué esa metáfora es tan precisa: porque la diferencia entre la memoria de tu programa y un archivo en disco es exactamente la diferencia entre poner un papel sobre el escritorio y guardarlo en un cajón.

Alan Kay y su equipo desarrollaron entre 1970 y 1973 el sistema **Smalltalk**, el primero en combinar ventanas, iconos, ratón y menús —lo que hoy conocemos como interfaz gráfica. Apple llevó esas ideas al mercado masivo con el Macintosh en 1978; Microsoft con Windows en 1985.

Antes de Smalltalk, los archivos se llamaban *datasets* y se referenciaban por códigos internos. El término *file* fue elegido deliberadamente para evocar los archiveros físicos de oficina.

Referencia: Kay, A. & Goldberg, A. (1977). "Personal Dynamic Media." *IEEE Computer*, 10(3), 31–41.

Memoria vs. archivos: el escritorio y los cajones

Cuando tu programa declara una variable, la computadora le asigna espacio en la **memoria RAM**. Puedes leer ese valor, modificarlo, usarlo en cálculos. Es rápido y cómodo, como tener un papel justo frente a ti sobre el escritorio.

El problema es que el escritorio se limpia solo cuando el programa termina. Todas las variables, todos los arreglos, todos los cálculos que hiciste desaparecen. La próxima vez que ejecutes el programa, empiezas de cero.

Los **archivos** resuelven esto. Un archivo vive en el disco duro (o en un SSD), no en RAM. Cuando el programa termina, el archivo sigue ahí. Cuando apagas la computadora, el archivo sigue ahí. Es como guardar un papel en un cajón: el cajón no se vacía solo.

Característica	Memoria (RAM)	Archivo (disco)
Acceso	Instantáneo	Más lento
Persiste al cerrar	No	Sí
Persiste al apagar	No	Sí
Capacidad típica	Gigabytes	Terabytes
Metáfora	Escritorio	Cajón

✓ Recuerda

Un programa puede guardar resultados en un archivo y otro programa puede leerlos después. Esa es la base de todo sistema de persistencia: bases de datos, configuraciones, imágenes, música, documentos. Todo es, al final, un archivo.

¿Qué es un archivo, exactamente?

Un archivo es una **secuencia de bytes** almacenada en disco. Nada más. No tiene estructura mágica, no sabe qué contiene, no se protege solo. Lo que hace que un archivo sea texto, imagen o música es únicamente cómo decidimos interpretar esos bytes.

• Verlo con tus propios ojos

Antes de escribir una sola línea de C++, crea un archivo de texto a mano:

Windows Clic derecho en el escritorio → Nuevo → Documento de texto. Escribe algunas letras (por ejemplo, Hola) y guarda.

Linux / Mac En la terminal: `touch prueba.txt`, luego ábrelo con cualquier editor y escribe lo mismo.

Ahora ábrelo con un **editor hexadecimal en línea** (por ejemplo, hexed.it). Lo que ves es el contenido real del archivo: cada carácter representado por su valor en hexadecimal.

La H de "Hola" es 0x48, la o es 0x6F, la l es 0x6C, la a es 0x61. Son los valores de la tabla ASCII que ya conoces.

Carácter	Decimal	Hexadecimal	Binario
H	72	0x48	01001000
o	111	0x6F	01101111
l	108	0x6C	01101100
a	97	0x61	01100001

Fíjate en el tamaño del archivo: 4 bytes para 4 caracteres. Agrega una letra más y el archivo crece exactamente 1 byte.

En C y C++, el estándar garantiza que `sizeof(char) == 1` en cualquier plataforma, compilador o arquitectura. Un `char` siempre ocupa exactamente 1 byte. Por

eso un archivo de texto con n caracteres ocupa exactamente n bytes (más los saltos de línea).

Curiosidad histórica: el primer disco duro comercial, el IBM RAMAC 350 (1956), almacenaba 5 MB, pesaba una tonelada y se rentaba por USD 3 200 al mes. Una microSD actual de 1 TB cuesta menos de USD 100 y cabe en la uña del pulgar. El mismo concepto de “archivo en disco”, 200 000 veces más capacidad.

Referencia: IBM Archives. “IBM 350 disk storage unit.” ibm.com/ibm/history/exhibits/stor

• Saltos de línea

Un salto de línea también es un carácter (o dos). Aquí hay una diferencia importante entre sistemas operativos:

Sistema	Secuencia	Bytes
Linux / Mac	\n	0x0A (1 byte)
Windows	\r\n	0x0D 0x0A (2 bytes)

Si creas un archivo de texto en Windows y lo abres en Linux (o viceversa), puedes ver caracteres extraños al final de cada línea. Esto es causa de muchos bugs cuando se comparten archivos entre sistemas. La biblioteca <fstream> en modo texto maneja la conversión automáticamente en la mayoría de los casos, pero es bueno saber que el problema existe.

La biblioteca <fstream>

Para leer y escribir archivos en C++ se usa la biblioteca <fstream> (*file stream*). Provee tres clases:

Clase	Uso	Analogía
<code>ifstream</code>	Leer de un archivo	Sacar un papel del cajón
<code>ofstream</code>	Escribir en un archivo	Meter un papel al cajón
<code>fstream</code>	Leer y escribir	Cajón de doble apertura

La idea central es que un archivo se comporta igual que `cin` y `cout`: usas los mismos operadores `>` y `<`, solo que aplicados a un objeto que representa un archivo en disco en lugar de la terminal.

Si ya sabes usar `cin` y `cout`, ya sabes casi todo lo necesario para leer y escribir archivos. `cin` y `cout` son también **streams**: el primero conectado al teclado, el segundo a la pantalla. `ifstream` y `ofstream` son lo mismo pero conectados a un archivo en disco. El operador cambia de destino, no de sintaxis.

Leyendo un archivo línea por línea

El primer ejemplo lee un archivo de texto y muestra su contenido en pantalla.

canciones.txt:

```
Walking on Sunshine
Happy
Color Esperanza
```

archivos1.cpp:

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main(){
    ifstream ifile("canciones.txt");
    string cancion;

    while (getline(ifile, cancion)){
        cout << cancion << endl;
    }
}
```

Desglose:

`ifstream ifile("canciones.txt");` Crea un objeto de tipo `ifstream` y lo conecta al archivo `canciones.txt`. Si el archivo no existe, el objeto queda en estado de error pero el programa no truena por defecto.

`while (getline(ifile, cancion))` Lee una línea completa del archivo y la almacena en `cancion`. El salto de línea se consume pero *no* se incluye en la cadena. El `while` se detiene cuando no quedan más líneas: `getline` devuelve el propio stream, y al llegar al final del archivo el stream se evalúa como `false`.

`cout << cancion << endl;` Muestra la línea en pantalla. El `endl` agrega el salto que `getline` ya consumió del archivo.

El archivo debe estar en el mismo directorio que el **ejecutable** (no necesariamente donde está el `.cpp`). Si el archivo no se encuentra, el `while` no itera nunca y el programa termina en silencio. Para detectarlo:

```
ifstream ifile("canciones.txt");
if (!ifile){
    cout << "Error: no se pudo abrir el archivo" << endl;
    return 1;
}
```

Escribiendo en un archivo

Ahora al revés: en lugar de leer desde un archivo, escribimos resultados en uno. **archivos2.cpp**:

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main(){
    ofstream ofile("resultados.txt");
    int x = 10;
    int y = 20;

    ofile << x+y << " ";
    ofile << y-x << " ";
    ofile << (x+y)/2 << endl;

    ofile.close();
}
```

Después de ejecutar este programa, `resultados.txt` contendrá:

```
30 10 15
```

El operador `<<` es exactamente el mismo que usas con `cout`. La única diferencia es que el destino es un archivo en lugar de la pantalla.

• ¿Por qué llamar a `close()`?

El sistema operativo usa un **buffer**: acumula los datos antes de escribirlos al disco para hacer menos operaciones físicas, que son lentas. Al llamar `close()`, le dices que vacíe el buffer y garantice que todo llegó al disco. Si el programa termina sin cerrar, el buffer se vacía automáticamente, pero llamar `close()` explícitamente es buena práctica.

✓ Recuerda

Si `resultados.txt` ya existía, `ofstream` lo **sobreescribe** desde el principio. Si quieres *agregar* contenido al final sin borrar lo anterior, usa el modo `fstream::app` (*append*):

```
ofstream ofile("resultados.txt", fstream::app);
```

Leyendo valores numéricos

El programa anterior escribió tres números en `resultados.txt`. Ahora los leemos con otro programa.

archivos3.cpp:

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main(){
    ifstream fresultados("resultados.txt");
    int x, y, z;
    fresultados >> x >> y >> z;
    cout << "La suma de " << x << " " << y << " y " << z;
    cout << " es " << x+y+z << endl;
}
```

El operador `>>` sobre un `ifstream` funciona igual que sobre `cin`: salta espacios y saltos de línea, lee el siguiente token y lo convierte al tipo indicado.

Las tres lecturas se pueden escribir separadas o encadenadas:

```
fresultados >> x;
fresultados >> y;
fresultados >> z;
// equivale a:
fresultados >> x >> y >> z;
```

Cada `>>` devuelve el stream, lo que permite el encadenamiento. Con `cin` lo haces todo el tiempo sin pensarlo.

Nota el flujo completo de los tres ejemplos: **archivos2** *escribe* resultados.txt; **archivos3** lo *lee* desde otro programa. Ese es el poder de los archivos: la comunicación entre programas, sesiones y usuarios distintos.

Actividad: agenda de cumpleaños

Vamos a construir una pequeña base de datos colaborativa. Cada persona genera un archivo con su nombre y fecha de cumpleaños, lo comparte con el grupo, y entre todos escriben programas para consultarla.

- **Formato del archivo**

Cada archivo personal tiene cuatro líneas, en este orden:

```
nombre
apellidos
día
mes
```


Por ejemplo, el archivo de Hannah Fabián (cumpleaños el 29 de febrero) sería:

```
Hannah  
Fabián  
29  
2
```

• Parte A: genera tu archivo (10 min)

Escribe un programa que genere tu propio archivo usando lo que aprendiste en **archivos2.cpp**. Nómbralo con tu nombre en minúsculas y sin espacios, por ejemplo `hannah.txt`. Una vez generado, súbelo a la carpeta de Google Drive indicada por el profesor.

• Parte B: las consultas (40 min)

Descarga todos los archivos de tus compañeros y colócalos en una carpeta. Luego resuelve los siguientes problemas. Para cada uno escribe un programa separado.

1. Unir todos los datos en un solo archivo.

Lee cada archivo individual y escribe toda la información en un archivo llamado `agenda.txt`, una persona por línea con el formato:

```
nombre apellidos día mes
```

Puedes apoyarte en un archivo de texto auxiliar que liste los nombres de todos los archivos individuales, uno por línea.

2. ¿Quién cumple años en un mes dado?

Lee `agenda.txt`, pide un número de mes al usuario (1 = enero, 12 = diciembre) y muestra el nombre completo de todas las personas que cumplen años ese mes.

3. ¿Cuál es el siguiente cumpleaños a partir de hoy?

Hoy es 6 de mayo. ¿De quién es el próximo cumpleaños? Si ningún compañero cumple años después del 6 de mayo, el siguiente es el primero del año que viene. Si varias personas comparten la fecha, muéstralas todas.

4. ¿Qué par de compañeros tiene los cumpleaños más cercanos?

Encuentra el par de personas cuyas fechas de cumpleaños están separadas por el menor número de días (considera el año como circular: el 31 de diciembre y el 1 de enero están separados por 1 día).

Ejercicios

1. Escribe un programa que pida al usuario tres calificaciones, las guarde en un archivo llamado `calificaciones.txt` (una por línea) y luego lea ese mismo archivo y muestre el promedio.
2. El programa del ejercicio anterior sobrescribe el archivo cada vez que se ejecuta. Modifícalo para que *agregue* las calificaciones al final sin borrar las anteriores. Después de agregar, lee todo el archivo y calcula el promedio de *todas* las calificaciones almacenadas.

3. Escribe un programa que lea `canciones.txt` y genere un archivo llamado `canciones_numerada` donde cada línea tenga el número de línea seguido de la canción:

```
1. Walking on Sunshine
2. Happy
3. Color Esperanza
```

4. Escribe un programa que lea dos archivos de texto (cuyos nombres pide al usuario) y genere un tercer archivo que contenga el contenido del primero seguido del contenido del segundo.
5. ¿Qué ocurre si intentas leer de un archivo que no existe con `ifstream`? ¿Lanza una excepción? ¿Da error de compilación? ¿Simplemente no lee nada? Primero razona la respuesta, luego verifica experimentalmente.

Resumen

- La **memoria RAM** es volátil: los datos desaparecen cuando el programa termina. Los **archivos** en disco persisten.
- Un archivo es una secuencia de bytes. En texto, cada carácter ocupa exactamente 1 byte (valor ASCII).
- `<fstream>` provee `ifstream` (leer), `ofstream` (escribir) y `fstream` (ambos).
- Los operadores `>>` y `<<` funcionan igual con archivos que con `cin` y `cout`: misma sintaxis, distinto destino.
- `getline(stream, cadena)` lee una línea completa; el `while` itera hasta el final del archivo.
- `ofstream` sobrescribe el archivo por defecto; usa `fstream::app` para agregar al final.
- Llama a `close()` al terminar de escribir para garantizar que el buffer se vacíe a disco.
- En Windows los saltos de línea son `\r\n` (2 bytes); en Linux/Mac son `\n` (1 byte).